

다이소 AI 상품 위치 안내 - 배포 및 운영 가이드

1. 전체 서버 구성

다이소 AI 상품 위치 안내 시스템은 **Lightsail** 가상 서버 두 대(서버 A, 서버 B)로 구성되며, 각 서버에 **Docker Compose**로 서비스를 분리해 배포합니다. 서버 A는 **애플리케이션 서버**로서 API 게이트웨이와 음성 STT 처리, 검색 API, 검색 결과 QR 생성, 캐시 제어 등의 역할을 담당합니다 ①. 서버 B는 **데이터 서버**로서 검색 백엔드 구성요소들을 포함하며, **BM25** 기반 검색엔진(Elasticsearch), **벡터 임베딩** 검색 DB(Qdrant), 그리고 **캐시/세션 저장소**(Redis)를 실행합니다 ②. 두 서버는 네트워크로 연결되어 서버 A의 애플리케이션이 서버 B의 Elasticsearch/Qdrant/Redis에 접근하여 검색 작업을 수행합니다.

서버 A에는 애플리케이션 컨테이너(FastAPI 기반)와 Next.js 프론트엔드 컨테이너가 Docker로 올라가며, **Nginx**를 호스트 수준에서 설치해 80/443 포트 트래픽을 받아 내부 API(8000번)와 프론트엔드(3000번)로 리버스 프록시합니다 ③ ④. 서버 B에는 Docker Compose를 통해 Elasticsearch(9200), Qdrant(6333), Redis(6379) 컨테이너가 실행되고, 서버 A 외에는 접근이 제한됩니다 ⑤. 운영 시 보안을 위해 서버 B의 검색 서비스 포트(9200, 6333, 6379)는 **방화벽(UFW)** 설정으로 서버 A의 IP만 허용하는 것이 권장됩니다 ⑤.

요약하면, **서버 A**는 클라이언트로부터의 **요청을 처리하는 API 게이트웨이**이며 음성 WebSocket/STT, 자연어 이해, 하이드브리드 검색 로직, 결과 반환 및 QR 생성까지 모든 애플리케이션 로직을 수행합니다. **서버 B**는 서버 A의 검색 요청에 응답하는 **데이터 레이어**로, 텍스트 BM25 검색은 Elasticsearch가, 임베딩 벡터 검색은 Qdrant가 담당하고, **검색 결과 캐싱**과 세션 관리는 Redis가 담당합니다 ②. 두 서버 모두 AWS Lightsail 환경에서 구동되며, Docker Compose를 통해 컨테이너 단위로 서비스가 관리됩니다.

2. 배포 절차 및 Git 관리

운영 환경에 새로운 코드를 배포할 때는 **Git**을 통해 버전 관리를 하고, Docker Compose로 컨테이너를 재빌드/재시작하는 순서로 진행합니다. 기본 절차는 다음과 같습니다:

1. **코드 업데이트**: 서버 A에서 운영 Git 브랜치(`main` 또는 지정된 릴리스 태그)를 `git pull`로 최신 상태로 갱신합니다. 운영 안정 버전은 Git **태그**를 활용하여 관리하며, 예를 들어 `v0.2` 또는 `v0.3-safe` 등의 태그로 특정 릴리스 커밋을 표시해 둡니다. 배포 시에는 가급적 이러한 태그를 체크아웃하여 일관된 버전을 사용할 것을 권장합니다 (예: `git fetch && git checkout tags/v0.3-safe`). 이렇게 하면 운영 중인 버전들을 태그 (예: `v0.2`, `v0.3-safe` 등)로 구분하여 추적할 수 있습니다.
2. **Docker 이미지 빌드**: 코드가 업데이트되었으면, Docker Compose를 통해 서버 A의 애플리케이션 및 프론트엔드 이미지를 빌드합니다. 예를 들어 서버 A에서는 프로젝트 디렉토리에서 다음 명령으로 이미지를 빌드 및 컨테이너를 기동합니다:

```
docker compose -f infra/docker-compose.app.yml up -d --build
```

이 명령은 변경된 코드를 반영하여 `daiso-api` (FastAPI)와 `daiso-frontend` 이미지를 빌드하고 컨테이너를 백그라운드로 재시작합니다 ⑥ ⑦. 빌드 과정에서 Python 패키지 요구사항 설치 및 프론트엔드 번들링이 수행되며, 완료 후 새로운 컨테이너가 실행됩니다.

3. **서버 B 데이터 서비스 재기동 (필요 시)**: 서버 B의 Elasticsearch/Qdrant/Redis 등 데이터 서비스는 일반적으로 코드 업데이트의 영향을 직접 받지 않지만, 구성 변경이나 버전 업데이트가 있었다면 Docker Compose로 다시 올려야 합니다. 서버 B에 변경이 없다면 재시작은 필요 없으며, 변경 사항이 있을 경우 `docker compose -f infra/docker-compose.data.yml up -d`로 필요한 서비스를 다시 시작합니다. Lightsail 메모리나 설정 변경으로 데이터 서비스를 재시작해야 할 때는 서버 B에서 제공하는 스크립트를 사용할 수 있습니다 (`infra/start_data.sh`). 예를 들어:

```
# 서버 B에서
./infra/start_data.sh down # 데이터 서비스 중지
./infra/start_data.sh      # 데이터 서비스 다시 시작 (백그라운드 실행)
```

`start_data.sh` 스크립트는 Elasticsearch 메모리 관련 커널 파라미터 설정(`vm.max_map_count`) 적용 후 Compose를 실행해주며, logs나 status 옵션으로 모니터링도 지원합니다 ⁸ ⁹.

4. **헬스체크 및 서비스 확인**: 서버 A와 B의 컨테이너가 모두 기동된 후, 아래와 같은 건강 확인을 수행합니다:

5. **Elasticsearch 상태**: `curl http://<서버B IP>:9200/_cluster/health`로 클러스터 상태가 `"status": "green"` 또는 `"yellow"` 인지 확인 ¹⁰.
6. **Qdrant 상태**: `curl http://<서버B IP>:6333/healthz`로 `OK` 응답 확인 ¹⁰.
7. **Redis 상태**: 서버 A 또는 B에서 `redis-cli -h <서버B IP> ping`으로 `PONG` 응답 확인 ¹⁰.
8. **API 서버 상태**: `curl http://<서버A IP>/health`로 애플리케이션 서버가 `{"status": "healthy", ...}` 200 OK를 반환하는지 확인 ¹¹.
9. **프론트엔드 확인**: `curl http://<서버A IP>`로 프론트엔드 기본 페이지(정적 HTML)가 나오는지 확인합니다 ¹¹.
10. **전체 검색 시나리오 점검**: 아래 운영 점검 항목에 따라 `/v1/search` 및 `/ws/stt` 등을 테스트합니다.

이와 같은 순서로 **서버 B → 서버 A** 순으로 배포를 진행하면 안전합니다 ¹². 서버 B의 데이터 서비스가 먼저 준비되어 있어야 서버 A의 애플리케이션이 기동 시 Elasticsearch/Qdrant에 정상 연결되기 때문입니다. 만약 서버 B를 재시작한 경우, 서버 A의 애플리케이션이 이미 떠 있는 상태라면 내부에서 재시도 로직으로 연결을 확보하지만, 중요한 업데이트의 경우 서버 A 컨테이너도 재시작하여 초기화하는 것이 좋습니다.

또한, 긴급 수정 등의 이유로 빠르게 재배포할 때는 Git 운영 태그를 활용해 이전 버전으로 되돌리는 전략도 사용합니다. 예를 들어, v0.3 버전 배포 후 이슈 발생 시 `git checkout tags/v0.2`로 롤백한 뒤 이미지를 다시 빌드하여 (`up -d --build`) 신속히 이전 안정 버전으로 복구할 수 있습니다.

3. 운영 모드 설명 및 설정법

다중 검색 모드 및 안전 모드를 통해 검색 동작 및 외부 API 호출을 제어할 수 있습니다. 환경변수로 설정 가능한 주요 **운영 모드**는 다음과 같습니다:

- **검색 모드 (SEARCH_MODE)**: 검색엔진의 동작 방식을 지정합니다. 설정값으로 `"bm25_only"`, `"dense_only"`, `"hybrid"` 세 가지를 사용할 수 있습니다 ¹³.
- **bm25_only**: 키워드 BM25 검색만 수행합니다. Elasticsearch를 통해서만 결과를 검색하며 벡터 임베딩 검색은 시도하지 않습니다. Elasticsearch가 가동 중이면 BM25 결과를 반환하고, 만약 Elasticsearch가 다운된 경우 자동으로 로컬 SQLite 검색으로 폴백합니다 ¹⁴.
- **dense_only**: 벡터 임베딩 검색만 수행합니다. Qdrant를 통해 유사도 검색을 하고 BM25 검색은 제외합니다. 이 모드에서는 **질의 임베딩**을 위해 외부 임베딩 API 호출이 필요하므로, 이 설정을 사용할 때는

SAFE_MODE가 꺼져 있어야 제대로 동작합니다. Qdrant가 다운된 경우 자동으로 SQLite 검색으로 폴백하도록 동작합니다.

- **hybrid** (기본값): BM25와 임베딩 **하이브리드 검색**을 수행하여 두 결과를 **RRF 융합**(Rank Reciprocal Fusion)합니다 ¹⁵ ¹⁶. Elasticsearch와 Qdrant 둘 다 정상일 때는 각 결과를 산출한 후 랭킹 점수를 융합하여 최종 Top-K를 결정합니다. 둘 중 하나만 가용한 경우 해당 검색 방식만으로 결과를 반환하며, 둘 다 불가할 경우 SQLite 폴백을 사용합니다 ¹⁷.

노트: `SEARCH_MODE` 는 `.env`에 명시적으로 설정하지 않으면 기본값 `"auto"` 로 동작하며, 이는 **가능하면 hybrid 사용, 불가 시 자동 폴백**하는 로직과 동일합니다 ¹⁸. 일반적인 운영에서는 `SEARCH_MODE` 를 특별히 지정하지 않고 hybrid 모드를 사용합니다. 특정 상황(예: 성능 테스트나 비교 실험)에서만 `bm25_only`나 `dense_only`로 설정해볼 수 있습니다.

- **안전 모드** (`SAFE_MODE`): 외부 LLM 서비스 호출을 **차단**하는 모드입니다. 이 값을 `true` 또는 `1`로 설정하면 검색 파이프라인에서 **외부 AI API를 이용한 작업을 모두 비활성화**합니다 ¹⁹. 구체적으로, **자연어 이해(NLU) 모델 호출, 검색 쿼리 확장(키워드 확장) 호출, LLM 리랭킹 호출**을 하지 않고 **로컬 휴리스틱 또는 캐시 결과만** 사용합니다 ²⁰ ¹⁹. `SAFE_MODE`에서는 예를 들어 Gemini와 같은 LLM API 키가 있어도 호출하지 않으며, NLU는 정규식 기반 로컬 분석으로 대체되고, 검색어 확장은 빈 결과 또는 사전 정의 룰만 사용합니다 ²¹ ²². 리랭킹도 외부 LLM 없이 내부 점수 또는 룰 기반으로 진행됩니다.

이러한 모드 설정은 **환경변수**로 제어되며, 운영 시 `.env` 파일에서 값을 세팅하고 서버 A 컨테이너를 재시작하면 적용됩니다. 예를 들어, `.env` 파일에 아래와 같이 설정할 수 있습니다 ²³:

```
SEARCH_MODE=bm25_only    # 검색 모드: bm25_only, dense_only, hybrid 중 선택
SAFE_MODE=1              # Safe 모드: 1로 설정 시 외부 LLM 호출 차단
```

위처럼 설정 후 서버 A의 컨테이너를 재시작하면 (`docker compose restart` 등) 해당 모드가 활성화됩니다. Safe 모드의 활성화 여부는 `/health` 응답의 `search_pipeline.flags.safe_mode` 값이나, `/v1/search` 결과의 `metadata.flags.safe_mode` 필드에서 확인할 수 있습니다 (True/False로 표시) ²⁴.

운영 모드별 동작 요약:

- Safe 모드 OFF + Hybrid 모드 = **풀기능 모드** (외부 LLM 통한 NLU/확장/리랭크 + BM25+벡터 융합 검색)
- Safe 모드 ON + Hybrid 모드 = **보수적 검색 모드** (로컬 NLU/확장/리랭크 + BM25+벡터 융합 검색) - 외부 API 호출 없음
- Safe 모드 ON + `BM25_only` = **로컬 BM25 전용 모드** (외부호출 없이 BM25만 사용, 실패시 SQLite)
- Safe 모드 ON + `Dense_only` = **로컬 벡터 전용 모드** (권장 않음: 임베딩 생성에 외부 호출 필요하므로 Safe+Dense 조합 시 실질적으로 결과 없음)
- Safe 모드 OFF + `BM25_only` = **BM25 전용 모드** (NLU/확장 등은 외부 이용, 검색은 BM25만)
- Safe 모드 OFF + `Dense_only` = **벡터 전용 모드** (NLU/확장 외부 이용, 검색은 임베딩만)

일반 운영에서는 **SAFE_MODE=false** (외부 API 활용)와 **SEARCH_MODE=hybrid** (하이브리드 검색) 조합으로 최고의 검색 품질을 내는 것을 목표로 합니다. 다만, 외부 API 비용이나 지연 문제가 우려될 때는 **SAFE_MODE**를 켜서 **벤더 호출 없이도 서비스가 동작**하도록 할 수 있습니다 ²⁰.

4. 필수 환경변수 목록

운영자가 관리해야 할 주요 **환경변수(.env)**와 설정값은 다음과 같습니다. (.env 파일은 Git에 커밋되지 않도록 `.gitignore` 설정으로 제외하며, 운영 서버 내에서만 보관합니다.)

• API 키 관련:

- `GOOGLE_API_KEY` - Google Cloud 의 Generative AI API 키 (예: Gemini 모델 사용). NLU 의도분석이나 임베딩 호출에 사용됩니다. 프로젝트 설정상 우선적으로 사용되는 키이며, 없다면 `GEMINI_API_KEY` 를 대체로 시도합니다 ²⁵ .
- `GEMINI_API_KEY` - 네이버 클라우드의 HyperCLOVA Gemini API 키. Google API 키가 없을 경우에만 사용됩니다. 두 키 중 하나는 설정되어야 외부 LLM 호출이 가능합니다. (SAFE_MODE가 true이면 설정돼 있어도 호출되지 않습니다.)
- `GOOGLE_APPLICATION_CREDENTIALS` - Google Cloud STT에 사용할 서비스계정 JSON 키 파일 경로입니다. Docker 컨테이너 내 경로를 가리키며, 예를 들어 `backend/daisoproject-sst.json` 파일 경로를 설정합니다. Docker Compose에서 해당 JSON을 `/run/secrets/` 경로로 마운트하고 이 변수에 경로를 넣어 Google STT 인증에 사용합니다 ²⁶ . 해당 파일에는 민감한 인증 정보가 있으므로 권한은 최소 (read-only)로 설정하고, 절대 Git 등에 올리지 않도록 주의해야 합니다.

• 검색 서비스 연결 설정:

- `QDRANT_URL` - Qdrant 벡터 DB의 HTTP 주소 (예: `http://<서버B IP>:6333`) ²⁷ . 서버 A에서 서버 B의 Qdrant에 접속하기 위한 URL입니다.
- `QDRANT_API_KEY` - (선택사항) Qdrant에 API Key 설정이 걸려있는 경우 입력. 기본 Qdrant 설정상 필요 없을 수 있습니다.
- `QDRANT_COLLECTION` - Qdrant에서 벡터를 저장하는 컬렉션명. 기본값 `products` 로 설정되어 있으며, 색인 및 검색 시 사용됩니다 ²⁸ .
- `ELASTIC_URL` - Elasticsearch의 URL (예: `http://<서버B IP>:9200`) ²⁹ . 서버 A에서 BM25 검색을 위해 사용.
- `ELASTIC_API_KEY` - (선택사항) Elasticsearch에 보안이 설정된 경우 API 키 또는 인증 토큰. 현재 구성 ³⁰에서는 xpack.security를 꺼두었으므로 빈 값으로 둡니다.
- `ELASTIC_INDEX` - Elasticsearch에서 사용할 인덱스명. 기본값 `products` 로, 상품 데이터 색인이 저장된 인덱스 이름입니다 ²⁹ .
- `REDIS_URL` - Redis 서버 접속 URI (예: `redis://<서버B IP>:6379/0`)입니다 ³¹ . 0번 DB를 사용하고, 비밀번호 없는 기본 포트 구성이라면 `redis://` 형식으로 설정합니다. 애플리케이션 기동 시 해당 변수로 Redis에 연결하며, 연결 성공 시 로그에 “Redis cache connected: <URI>”가 출력됩니다 ³² .

• 검색 동작 모드:

- `SEARCH_MODE` - 앞서 설명한 검색 모드 설정값 (`hybrid`, `bm25_only`, `dense_only`). 설정되지 않으면 auto로 간주합니다 ¹⁸ .
- `SAFE_MODE` - Safe 모드 토글 (`0` 또는 `1`). 1로 설정 시 외부 API 미사용 ¹⁹ .
- `DISABLE_HYBRID` - 하이브리드 검색 비활성화 플래그 (`0` 또는 `1`). 일반적으로 사용되지 않지만 1로 설정 시 Elasticsearch/Qdrant 대신 무조건 로컬 SQLite 검색으로 폴백합니다 ¹⁹ . 긴급시 전체 서버 B를 우회해야 할 경우 사용할 수 있습니다.
- `FORCE_SQLITE` - (내부옵션) 1로 설정 시 조건과 무관하게 검색에 SQLite만 사용하도록 강제하는 옵션입니다. 일반 운영에서는 사용하지 않습니다.

• 임베딩 및 검색 파라미터:

- `EMBEDDING_PROVIDER` - 임베딩 생성 서비스 제공자 (`google` 등). 현재 기본값은 Google의 Generative AI입니다 ³³.
- `EMBEDDING_MODEL` - 임베딩에 사용할 모델 이름 (예: `gemini-embedding-001`) ³³.
- `SEARCH_TOP_K_BM25`, `SEARCH_TOP_K_DENSE`, `SEARCH_TOP_K_FUSED` - BM25, Dense, 융합 검색 시 각각 가져올 상위 결과 개수 설정 ³⁴. 기본 BM25 30개, 벡터 30개를 검색 후 RRF 융합으로 최종 10개 (`SEARCH_TOP_K_FUSED`)를 사용합니다.
- `SEARCH_RRF_K` - RRF 융합 시 고려할 상위 문서 수 (기본 60개) ³⁴. BM25 30 + Dense 30 = 60개 결과를 융합.
- `SEARCH_FUSION_METHOD` - 융합 방법 (`rrf` 또는 `weighted`). 기본 RRF (Reciprocal Rank Fusion) 방식이며 `weighted`로 설정 시 가중치 평균 방식 융합도 가능 ³⁵.

• 리랭킹 및 벤더 설정:

- `RERANK_MODE` - 상품 검색 후 **재정렬 단계**에 사용할 모드 (`local`, `vendor`, `mock`, `simulated` 중 하나). 기본값은 `local`이며 로컬 키워드 기반 정렬을 합니다. `vendor`로 설정하면 외부 LLM(Gemini)에 의한 재정렬을 시도하며, `SAFE_MODE=true`이면 이 호출도 차단됩니다.
- `VENDOR_ENABLED` - 별도 벤더 (Gemini) 호출 토글. `false`일 경우 강제로 외부 LLM 호출을 하지 않으며, `true`이면 `RERANK_MODE`에 따라 호출합니다. (현재 구조상 `SAFE_MODE`로 대부분 대체 가능하여 의미는 중복됩니다.)
- 그 외 `VENDOR_SAMPLE_RATE`, `VENDOR_MAX_CALLS_PER_MIN` 등은 LLM 벤더 호출 빈도 제어용으로 사용됩니다. 기본값은 0으로 설정돼 있어 운영에서는 외부 호출을 안 하도록 해둔 상태입니다 ³⁶ ³⁷.

• 기타 설정:

- `CORS_ORIGIN` - 프론트엔드 웹 페이지 도메인 설정입니다. 키오스크 모드 웹앱이 API를 호출할 때 CORS 허용을 위해 필요합니다. 운영 환경의 서버 A IP 또는 도메인을 이 변수에 지정해야 웹소켓 및 API 호출이 정상 진행됩니다 ³⁸.
- `GOOGLE_APPLICATION_CREDENTIALS` - (앞서 언급) Google STT용 인증 JSON 경로. 서버 A 컨테이너에서 해당 파일이 존재하고 경로가 유효해야 음성 인식 호출이 성공합니다.

위 환경변수들은 **.env 파일**에 키=값 형식으로 저장하고 Docker Compose가 컨테이너에 주입합니다 ³⁹. 운영시 .env 파일은 **절대로 Git에 올리지 않고**, Git에는 `.env.example` 과 같이 예시만 존재합니다. 실제 운영 키는 배포 담당자만 알고 안전하게 관리해야 합니다. 또한 서버 B IP 등의 민감 정보도 .env에 있으므로, 해당 파일 권한을 `600` 등 최소 권한으로 설정해두는 것을 권장합니다.

권장 보안 설정: .env 파일 자체를 Git에서 무시(`.gitignore`)하고, API 키 값들은 운영 시 수동으로 입력하거나 CI/CD 시스템의 Secrets로 관리합니다. Google STT JSON 키 파일 역시 `secrets/` 경로에 두고 Docker 마운트로 사용하는데, 이 파일의 경로와 이름(`daisoproject-sst.json`)은 외부에 노출되지 않도록 합니다 ²⁶. 서버 B의 Elasticsearch, Qdrant, Redis는 방화벽(UFW)으로 외부 접근을 막고 오직 서버 A IP만 허용했습니다 ⁵. 이러한 설정을 준수하면 환경변수와 서비스 포트 노출로 인한 보안 위험을 최소화할 수 있습니다.

5. 배포 후 필수 점검 항목

코드 배포나 서버 재기동 후에는 아래 **체크리스트**를 따라 서비스가 정상 동작하는지 확인해야 합니다 ⁴⁰. 운영자는 다음 항목들을 순서대로 검증합니다:

- **헬스 체크 엔드포인트 확인:** 서버 A의 `/health` API를 호출하여 정상 응답을 받는지 확인합니다. 예를 들어:

```
curl http://<서버A IP>/health
```

예상 응답은 HTTP 200 상태 코드와 함께 JSON 본문에 `{"status": "healthy", ...}` 형태로 여러 상태 필드가 포함됩니다 ⁴¹. 특히 `redis_cache.status`가 `"healthy"`이고 `search_pipeline`이 `"ready"`로 표시되는지 확인합니다. `whisper_model` 필드에는 로드된 Whisper STT 모델 사이즈(예: base)가 나타납니다 ⁴². 이로써 **API 서버 구동 및 주요 구성요소 초기화 완료 여부**를 확인할 수 있습니다.

- **검색 API 기능 점검:** `/v1/search` 엔드포인트에 시나리오를 대표하는 **텍스트 검색 요청**을 보내 결과를 확인합니다. 예를 들어, JSON 본문에 `store_id`와 `query`를 포함하여 검색 요청을 합니다:

```
curl -X POST http://<서버A IP>/v1/search \
  -H "Content-Type: application/json" \
  -d '{"store_id": "store_001", "query": "치솔"}'
```

이 요청에 대해 HTTP 200과 함께 `SearchResponse` 형식의 JSON이 반환되어야 합니다. 응답 JSON에는 `top3` 필드에 3개 내외의 검색 결과가 리스트로 포함되며, 각 결과는 `name` (상품명), `category` (카테고리), `price` (가격), `score` (검색 점수), `source` (검색출처) 등의 정보를 가집니다 ⁴³. 또한 `intent` (예: "PRODUCT_SEARCH"), `is_in_scope` (true/false), `needs_clarification` (모호성 질문 여부) 등의 메타데이터도 포함됩니다 ⁴⁴ ⁴¹. 정상이라면 요청한 상품 키워드에 대응하는 `top3` 결과 리스트가 의미 있게 채워져 나옵니다 ¹¹. 만약 `top3`가 빈 배열이거나 `is_in_scope=false`로 나오는 경우, 색인 데이터 누락이나 검색엔진 이상을 의심해야 합니다.

- **STT WebSocket 기능 점검:** 음성 입력을 처리하는 `/ws/stt` **웹소켓 엔드포인트**에 연결하여 실시간 STT가 동작하는지 확인합니다. 방법: 서버 A에서 WebSocket 클라이언트를 사용하여 `ws://<서버A IP>/ws/stt`에 연결합니다. CLI에서는 `wscat` 도구를 사용할 수 있습니다:

```
npm install -gwscat
wscat -c ws://<서버A IP>/ws/stt
```

연결 후 마이크 음성을 바이너리 스트림으로 전송하면, 서버로부터 인식된 텍스트 결과를 실시간으로 받을 수 있습니다. 수동 테스트가 어려운 경우, 브라우저에서 프론트엔드 키오스크 모드 페이지를 열어 실제 마이크로 음성을 입력해보는 것도 방법입니다. **연결 자체가 거부되지 않고** (Handshake 101 Switching Protocol OK) 곧바로 `"READY"` 혹은 초기 인식 결과 같은 메시지가 수신되면 WebSocket 연결은 정상입니다 ¹¹. 이후 음성을 말했을 때 텍스트 결과가 JSON으로 지속 수신되는지 확인하십시오. 만약 연결이 바로 닫히거나 응답이 없다면, STT 어댑터 초기화 문제나 Google STT 인증키 문제 가능성이 있으므로 로그를 확인해야 합니다.

- **Redis 캐시 동작 확인:** 검색 요청을 두 번 이상 보내보고 **캐시 적중 여부**를 확인합니다. 첫 번째 `/v1/search` 요청 시에는 캐시에 해당 질의가 없으므로 **MISS**가 발생하고, 두 번째로 동일한 질의를 요청하

면 **HIT**이 발생해야 합니다. 캐시 적중 여부는 서버 A 애플리케이션 **로그**에 남습니다. 예를 들어 로그에서:

```
[INFO] Cache HIT: keyword expansion for '치솔'
[INFO] Cache HIT: search results for ['치솔']
```

와 같은 메시지를 확인할 수 있습니다 ⁴⁵ ⁴⁶. 첫 요청 때는 **"Cache MISS"** (디폴트 로그레벨 debug)이 있을 것이며, 두 번째에는 위처럼 **HIT** 로그가 출력됩니다. 또한 `/health` 응답의 `redis_cache.connected`가 true인지, `used_memory_human` 등의 정보가 보이시지도 확인합니다 ⁴⁷. 캐시가 전혀 동작하지 않으면 Redis 연결 문제일 수 있으므로, 이 경우 `.env`의 `REDIS_URL` 설정이나 서버 B의 Redis 컨테이너 상태를 재확인합니다.

- **로그 모니터링**: 배포 후 잠시 동안은 서버 A의 애플리케이션 로그를 지속적으로 모니터링해야 합니다. 특히 **token_usage**(토큰 사용량), **latency**(지연 시간), **safe_mode 적용 여부** 등의 정보가 로그나 응답 메타데이터에 나타나므로 이상이 없는지 봅니다. 예를 들어 `/v1/search` 응답 JSON의 `metadata.flags.safe_mode`가 의도한 값과 일치하는지 (운영자가 `SAFE_MODE` 설정에 따라 true/false), `timing_ms.total`이 과도하게 높지는 않은지 등을 확인합니다. 로그 레벨이 debug인 경우, **요청 별 처리 시간**이나 **토큰 사용량**이 로그로 출력되기도 합니다. 정상적인 경우 Safe 모드가 꺼져 있으면 NLU/확장 단계에서 수십 또는 수백 토큰 정도 사용되고, Safe 모드이면 `"token_usage": {"total_tokens": 0}`로 나타납니다 ⁴⁸ ⁴⁹. **latency**의 경우 `timing_ms`에 `total`이 몇백 ms 내외인지 확인하고, 지나치게 (예: 수 초 이상) 높다면 어디서 지연이 생겼는지 추가 점검이 필요합니다.

요약하면, 배포 후에는 **헬스 체크 -> 기능 테스트(검색/음성) -> 캐시 확인 -> 로그 점검** 순으로 진행하면 됩니다. 아래 표는 주요 체크 항목과 기대되는 정상 동작 결과를 정리한 것입니다:

체크 항목	확인 방법 (명령 또는 행위)	기대 결과
API 헬스 체크	<code>curl http://<서버A>/health</code>	<code>status: healthy</code> 표시 ⁴¹
검색 API 테스트	<code>POST /v1/search</code> (샘플 질의)	<code>top3</code> 에 상품 결과 반환 ¹¹
웹소켓 STT 테스트	WebSocket <code>/ws/stt</code> 접속 및 음성	텍스트 실시간 전송 (응답 있음)
Redis 캐시 Ping	<code>redis-cli -h <서버B> ping</code>	<code>PONG</code> 응답
Redis 캐시 로그	서버 A 로그 모니터	2회째 검색 시 <code>Cache HIT</code> 로그 ⁴⁶
토큰 사용량	서버 A 로그 또는 응답 메타데이터	Safe 모드 시 0, 아니면 일정 값
응답 시간	서버 A 로그 또는 <code>timing_ms.total</code>	일반 질의 수백ms 이내

이상의 항목들을 모두 만족하면 배포가 성공적으로 이루어져 운영을 계속할 준비가 된 것입니다.

6. 검색 모드별 테스트

운영 중 특정 검색 모드의 성능이나 동작을 비교하고 싶다면, **모드를 달리하여 테스트**를 진행할 수 있습니다. 예를 들어 **BM25 전용 모드와 Hybrid 모드의 결과 비교** 테스트를 아래와 같은 절차로 수행합니다:

1. **BM25-only 모드 설정**: 서버 A의 `.env`에서 `SEARCH_MODE=bm25_only`로 설정하고 서버를 재시작합니다 (또는 일시적으로 환경변수를 override하여 프로세스 실행). Safe 모드 여부는 테스트 목적에 따라 on/

off 할 수 있으나, 공정한 비교를 위해 Hybrid 모드 테스트 시와 동일하게 유지합니다. 재시작 후 `/health`로 `search_pipeline.search_mode`가 "bm25_only"인지 확인합니다.

2. **테스트 질의 선정:** 한두 개의 대표 질의를 정합니다. (예: "물티슈", "빨래 건조대" 등 다양한 난이도의 질의 ⁵⁰.) 가능하면 일반 검색 질의와 모호한 질의 둘 다 선택하여 모드별 차이를 관찰합니다.

3. **BM25-only 결과 확인:** 선택한 질의로 `/v1/search` 요청을 보내 응답을 저장합니다. `top3` 결과의 `score` 및 `source` 필드에 주목합니다. BM25 전용 모드에서는 모든 결과의 `source`가 "bm25"로 표시되고, `score`는 BM25 점수입니다 ⁴³. 예를 들어 `top3[0].source = "bm25"`이며, `score`는 Elasticsearch에서 산출한 relevancy 점수입니다. 응답 `metadata.search.mode`도 "hybrid" 대신 "bm25_only"로 나타납니다.

4. **Hybrid 모드 결과 확인:** 서버 A를 `SEARCH_MODE=hybrid`로 변경 후 재시작합니다 (Safe 모드는 동일하게 설정). 같은 질의로 `/v1/search` 요청을 보내 응답을 저장합니다. hybrid 모드의 결과 `top3`는 BM25와 벡터검색 결과가 융합된 것이므로, `source` 필드가 혼합되어 있을 수 있습니다. 예를 들어 어떤 결과는 "bm25", 어떤 것은 "dense"로 표기될 수 있습니다 (각 결과가 BM25에서 온 것인지, 임베딩 유사도로부터 온 것인지 표시) ⁴³. 또한 `score` 값의 범위나 분포가 BM25-only때와 다를 수 있습니다(융합 점수는 정규화된 상대점수일 가능성이 높음). `metadata.search.mode`는 "hybrid"로 표시됩니다.

5. **결과 비교:** 두 모드의 `top3` 리스트를 비교하여 **결과 구성의 차이**를 분석합니다. BM25_only 모드에서는 키워드 일치에 강한 결과가 주로 나오고, Hybrid 모드에서는 임베딩을 통한 유사 의미 결과도 포함될 수 있습니다. 예를 들어 질의어에 오타나 유의어가 있을 경우 hybrid 모드가 더 관련 상품을 찾아낼 수 있고, dense_only 모드였다면 BM25에서는 못 찾을 결과를 내는지를 관찰합니다. 또한 **score**와 **source** 필드를 함께 검토하여, hybrid 모드에서는 상위 결과 중 일부가 "dense" 소스(벡터 기반)이며 BM25 점수가 낮았더라도 임베딩 유사도로 올라온 사례가 있는지 확인합니다.

6. **dense_only 모드 (선택):** 추가로 임베딩 검색 단독 효과를 보고 싶다면 `SEARCH_MODE=dense_only`로 설정 후 테스트할 수 있습니다. 이 경우 결과 `source`는 모두 "dense"로 표시되고 BM25 점수는 사용되지 않습니다. 다만, dense_only는 query를 벡터화해야 하므로 `SAFE_MODE`를 꺼야 의미 있는 결과를 얻습니다. dense_only 모드의 결과와 bm25_only 결과를 비교하면, 임베딩 검색이 **의미 매칭**을 얼마나 보완해주는지 파악할 수 있습니다.

7. **모드 전환 부하 및 캐시 유의:** 각 모드로 전환할 때 Redis 캐시에는 이전 모드의 검색 결과가 남아 있을 수 있으므로 정확한 테스트를 위해 **캐시를 비우는 것이 좋습니다**. `/cache` DELETE 엔드포인트를 호출하여 캐시를 초기화하거나, Redis CLI로 `FLUSHDB`를 수행합니다. 이렇게 해야 모드 변경 후 첫 요청이 항상 fresh 검색을 하게 되어 올바른 비교가 가능합니다.

위 테스트를 통해 **BM25 vs Hybrid vs Dense** 검색의 응답 차이를 정량/정성적으로 평가할 수 있습니다. 예를 들어, hybrid 모드에서 BM25 상위에는 없던 추가 연관 상품이 들어오거나 모호한 질의에 대해 `needs_clarification`이 true로 나오는지 등을 확인하게 됩니다. 이러한 모드 비교 결과는 검색 품질 튜닝이나 문제 발생 시 디버깅에 활용될 수 있습니다.

7. 장애 대응 시나리오

운영 중 다음과 같은 **부분 장애 상황**이 발생할 수 있으며, 시스템은 일부 자동 폴백을 제공하지만 운영자는 원인 파악 및 대응 조치를 알아두어야 합니다:

- **Elasticsearch 장애 (BM25 엔진 다운):** 서버 B의 Elasticsearch 컨테이너가 중지되거나 응답하지 않는 경우입니다. 이때 서버 A의 검색 파이프라인(하이브리드 모드 기준)은 자동으로 **BM25 부분을 제외**하고 벡터 검색

(Qdrant) 결과만으로 응답하려 시도합니다 51 17. HybridSearchService는 health-check에서 Elasticsearch가 불응이면 내부적으로 empty 리스트를 받고, Qdrant 결과만 활용하여 fused 결과를 만들거나 (사실상 dense_only와 유사) 둘 다 실패 시 최종적으로 SQLite 폴백을 수행합니다 17. 따라서 **일시적으로**는 사용자가 검색을 시도해도 벡터 기반 결과나 로컬 검색 결과가 제공되어 서비스가 완전히 죽지 않습니다. 로그를 보면 "⚠ Hybrid search services not fully healthy: {...}" 경고나 [PATCH] Hybrid search failed → fallback to SQLite 등의 메시지가 출력되었는지 확인하세요 52 53. 운영자는 이러한 로그 감지 시 **Elasticsearch 컨테이너를 재시작**하거나 (docker compose -f ...data.yml restart elasticsearch) 서버 B를 점검하여 신속히 복구해야 합니다. 복구 전까지는 검색 품질이 평소보다 떨어질 수 있음을 인지해야 합니다.

- **Qdrant 장애 (벡터 엔진 다운):** Elasticsearch 장애와 반대 시나리오입니다. Qdrant 컨테이너가 죽은 경우, hybrid 모드에서 벡터 검색 부분이 누락되어 BM25 결과만으로 검색이 동작합니다. 코드 상 HybridSearchService는 Qdrant health-check 실패 시 dense_results를 빈 리스트로 처리하고 BM25 결과만 반환합니다 54 17. 사용자는 키워드 기반 검색은 계속 사용할 수 있지만 의미벡터 보강이 안 되므로 성능이 저하됩니다. 이 역시 로그에 "Hybrid search services not fully healthy" 경고가 남을 수 있으며, 운영자는 **Qdrant 컨테이너 재기동**(docker compose restart qdrant)이나 서버 B 재확인을 통해 조치를 취합니다.

- **Redis 장애 (캐시 서버 다운):** Redis가 작동하지 않으면 검색 파이프라인은 캐시를 사용하지 않고 매번 풀 검색을 수행합니다. 다행히 코드에 **그레이스풀 디그라데이션**이 구현되어 있어, Redis 연결 실패 시 _get_redis()가 None을 반환하고 캐시 함수들이 **no-op**로 동작합니다 55 56. 즉, 서비스가 멈추진 않고 응답 시간만 조금 늘어날 뿐입니다. 로그에 "⚠ Redis not reachable – cache disabled: ..." 경고가 뜰 수 있으며, /health의 redis_cache.status가 "unavailable"로 표시됩니다 57. 운영자는 이때 **Redis 컨테이너를 재시작**하고 (server B: docker compose restart redis 또는 ./infra/start_data.sh 활용) 방화벽이 올바른지(서버 A->Redis 포트 허용) 점검합니다. Redis 복구 후에는 애플리케이션 로그에 "☑ Redis cache connected" 메시지 32가 다시 나타나야 정상입니다.

- **Google STT 인증 오류:** 음성 인식 기능에서 Google Cloud STT API 호출이 실패하는 경우입니다. 원인은 GOOGLE_APPLICATION_CREDENTIALS 경로나 파일 권한 문제, 혹은 GCP 할당량 소진 등이 될 수 있습니다. 이 상황에서 시스템은 **Whisper 모델을 사용한 폴백 STT**를 시도합니다 58. config에 fallback.enabled: true로 설정되어 있고, WebSocket STT 처리 모듈에서 Google 스트리밍 실패 시 Whisper base 모델로 자동 전환하게 되어 있습니다 58 59. 따라서 STT 호출 오류 발생 직후에도 사용자는 약간 지연될 뿐 계속 음성 인식 서비스를 이용할 수 있습니다 (Whisper는 로컬연산이라 다소 느릴 수 있음). 운영 로그에는 "Google STT failed, switching to Whisper" 류의 메시지가 나타날 수 있습니다. 이 경우 운영자는 **인증키 파일 경로와 권한**을 점검하고, GCP 콘솔에서 해당 서비스 계정이 STT API 사용 권한이 있는지 확인합니다. 문제가 해결되면 다음 WebSocket 연결부터는 다시 Google STT를 활용하게 됩니다. 만약 Whisper 폴백도 메모리 부족 등으로 실패하면 STT 응답이 빈 문자열로 돌아올 수 있으므로, 이때는 서버 A를 재기동하여 Whisper 모델을 다시 로드하거나 config.yaml의 모델 사이즈를 더 낮춰야 할 수도 있습니다 60.

- **전체 또는 다중 컴포넌트 장애:** 예를 들어 서버 B가 다운되어 Elasticsearch/Qdrant/Redis 모두 불응인 경우, 서버 A는 hybrid init 시도에서 모두 실패하여 **로컬 SQLite 폴백 모드**로 작동합니다 61 62. 이 상황에서는 검색 결과 품질이 크게 낮아지지만 최소한 상품명 문자열 매칭을 통해 서비스가 계속됩니다. 로그에 [PATCH] Hybrid search init failed, falling back to SQLite 같은 경고가 연속으로 뜰 것입니다 63 53. 이는 심각한 상태이므로 즉시 서버 B를 복구하고, 필요하면 서버 A도 재시작하여 hybrid 연결을 초기화해야 합니다.

- **기타 애플리케이션 오류:** NLU 벤더 API 에러, 리랭킹 벤더 에러 등은 Safe 모드 Off 상태에서 발생할 수 있습니다. 이때 통합 검색 파이프라인은 예외를 캐치하여 **fallback 처리**를 수행합니다. 예를 들어 Gemini LLM 호출이 실패하면 `is_fallback=true`로 표시하고 내부 keyword rerank로 대신하며, 응답 `error_type` 필드에 `"VENDOR_ERROR"` 등을 남깁니다⁶⁴. 운영자는 이러한 응답 필드를 모니터링하거나 로그의 예외 trace를 확인하여 외부 API 상태를 알 수 있습니다. 잦은 벤더 에러 발생 시 `SAFE_MODE`를 일시 활성화하여 서비스 지속성을 높이는 것도 고려합니다.

장애 조치 요약: 장애 발생 시 우선 서비스가 완전히 멈췄는지 여부를 파악합니다. 대부분의 검색 기능은 위 설명처럼 부분 동작하거나 폴백 모드로 돌아가기 때문에 “완전 중단”보다는 “성능 저하” 형태일 가능성이 높습니다. 운영자는 로그와 `/health` 등을 통해 원인을 추적한 뒤, **해당 컴포넌트를 재가동**하고 **구성요소별 재시작 순서**를 지켜야 합니다. 일반적으로 **데이터 레이어(B)를 먼저 복구**한 뒤 애플리케이션(A)을 필요 시 재시작하여 연결 고리를 복원합니다¹². Docker Compose로 모든 서비스를 한꺼번에 재시작해야 한다면, `docker compose down` → `docker compose up -d` 순으로 수행하되, 먼저 서버 B에서 내려 데이터 서비스를 모두 올린 뒤 서버 A를 올리는 순서를 지킵니다. 서버 A와 B 모두 재시작 후에는 앞서 **배포 후 체크리스트(섹션 5)**를 다시 한번 실행하여 정상 복구를 확인합니다.

8. CLI 명령어 예시

운영자가 자주 활용하게 될 **커맨드라인 인터페이스(CLI) 명령어**와 사용 예시는 아래와 같습니다. 이들은 서버 상태 점검, 기능 테스트, 로그 조회 등에 유용하며, 운영 가이드로서 미리 숙지해 둡니다:

- **검색 API 테스트 (curl):** REST API로 상품 검색을 호출하는 가장 간단한 방법은 `curl`을 사용하는 것입니다. 예를 들어, **지정한 매장(store_id)**에서 “물티슈”라는 질의를 검색하려면:

```
curl -X POST http://<서버A IP>/v1/search \
  -H "Content-Type: application/json" \
  -d '{"store_id": "store_001", "query": "물티슈"}'
```

위 명령은 JSON 본문으로 `store_id`와 `query`를 담아 POST 요청을 보냅니다. 응답으로 200 OK와 함께 `top3` 등의 필드가 포함된 JSON을 출력하며, 터미널에 바로 표기될 수 있습니다. 원하는 경우 `| jq` 등을 파이프로 연결해 포맷팅해서 볼 수도 있습니다. 이 `curl` 명령을 통해 프론트엔드 없이도 백엔드 검색 기능을 수시로 점검할 수 있습니다 (예: 배포 직후 정상 동작 확인 용도로).¹¹

- **STT WebSocket 테스트 (wscat):** WebSocket API `/ws/stt`는 브라우저를 통해서 주로 사용되지만, CLI에서도 `wscat` 도구를 활용하여 간단히 연결 테스트를 할 수 있습니다.

```
# wscat 설치 (Node.js 기반)
npm install -g wscat

# STT 웹소켓 연결 테스트
wscat -c ws://<서버A IP>/ws/stt
```

연결에 성공하면 프롬프트가 대기 상태로 변합니다. 여기서 오디오 데이터를 바이너리로 전송해야 실제 STT가 진행되지만, 단순히 연결 성공 (`Connected`) 메시지가 나오면 웹소켓 서비스가 열려있음을 확인할 수 있습니다. 좀 더 진전된 테스트를 하려면, 미리 준비한 PCM WAV 파일을 binary frame으로 전송하거나, 브라우저 콘솔에서 WebSocket을 열고 `send()`로 audio chunk를 보내는 등의 방법이 필요합니다. **간단 확인용으로**

는 위 연결 후 아무 키나 입력하지 않고 있으면 주기적으로 서버가 ping이나 응답을 보낼 수 있는데, 특정 타이밍 이후 연결이 유지되면 정상입니다. (혹은 연결 후 마이크로 짧게 말해보고 브라우저에서 결과를 확인.)

- **Docker 로그 확인:** Docker Compose로 관리되는 컨테이너 로그는 곧바로 애플리케이션 로그와 연결됩니다. 서버 A의 **API 서버 로그**를 보려면:

```
docker compose -f infra/docker-compose.app.yml logs -f api
```

명령을 사용합니다 7. `-f` 옵션은 follow로 실시간 로그를 계속 출력합니다. 프론트엔드의 경우 `logs -f daiso-frontend` (컨테이너 이름)로 보면 되고, 전체를 보려면 서비스 이름을 빼고 `logs -f` 하면 Compose에 정의된 모든 컨테이너 로그를 한 화면에 출력합니다. 서버 B의 경우, 제공된 스크립트를 사용하면 편리합니다:

```
# 서버 B에서
./infra/start_data.sh logs
```

이 명령으로 Elasticsearch, Qdrant, Redis 컨테이너의 현재 로그들을 실시간 출력해줍니다 8. 개별적으로 Docker 명령을 쓰려면 `docker compose -f infra/docker-compose.data.yml logs -f elasticsearch` 등으로 볼 수도 있습니다. 로그 확인을 통해 서비스의 현재 상태와 오류를 모니터링할 수 있으며, 운영 중 문제가 생기면 제일 먼저 이 로그들을 살펴봐야 합니다.

- **Docker 컴포즈 재시작:** 새로운 이미지 빌드 없이 **컨테이너만 재시작**하고 싶을 때 (예: 메모리 누수로 장기간 실행 후 메모리 초기화) 사용할 수 있는 명령어입니다.

- 서버 A 전체 재시작:

```
docker compose -f infra/docker-compose.app.yml restart
```

이것은 Compose 파일 내 모든 서비스 (`api`, `frontend`)를 재시작합니다 7. 컨테이너를 중지했다가 즉시 다시 시작하므로 다운타임이 수 초 내로 짧습니다.

- 서버 B 전체 재시작:

```
docker compose -f infra/docker-compose.data.yml down && \
docker compose -f infra/docker-compose.data.yml up -d
```

또는 간단히 `./infra/start_data.sh` 스크립트를 다시 실행해도 됩니다 65 66. 서버 B의 재시작은 Elasticsearch 초기화 시간 등을 고려하여 약간 더 시간이 걸릴 수 있습니다. **주의:** 서버 B를 재시작하면 서버 A에서 검색 호출 시 일시적으로 에러가 발생하거나 폴백모드로 작동할 수 있으므로, 가급적 트래픽이 적은 시간대에 수행합니다.

- 단일 서비스 재시작: 예를 들어 Elasticsearch만 재시작하려면 `docker compose -f infra/docker-compose.data.yml restart elasticsearch`와 같이 컨테이너 이름을 지정할 수 있습니다. Redis만 재시작도 유사하게 가능합니다.

- **애플리케이션 수동 실행 (디버그):** 필요에 따라 서버 A의 FastAPI 앱을 도커 외부에서 직접 실행해볼 수도 있습니다 (로컬 디버깅 등). 이 경우 Python 3.12 환경에서 `pip install -r requirements.txt` 설치 후 아래처럼 실행합니다:

```
export $(cat .env | xargs) # .env 환경변수 로드
uvicorn backend.main:app --reload --port 8000
```

이렇게 하면 개발 모드로 서버를 올릴 수 있습니다. 운영 환경에서는 권장되지 않지만 문제 원인 분석을 위해 일시적으로 써볼 수 있습니다. 이때도 .env 설정이 적용되므로 SAFE_MODE나 SEARCH_MODE 등의 효과를 비교 테스트할 때 유용합니다.

• 기타 명령어:

- **인덱스 구축:** 상품 DB를 처음 색인하거나 업데이트했을 때, 이를 Elasticsearch/Qdrant에 반영하려면 서버 A에서 인덱싱 스크립트를 실행합니다:

```
python -m backend.search.indexer --source sqlite
```

이를 실행하면 SQLite의 `products.db` 내용을 읽어 서버 B의 Elasticsearch 및 Qdrant에 색인을 수행합니다 ⁶⁷. 이 명령 전후로 서버 B의 Elasticsearch/Qdrant가 실행 중이어야 하며, .env에 해당 IP들이 설정되어 있어야 합니다. 성공 시 로그에 몇 개의 문서가 색인되었는지 출력됩니다.

- **메모리 모니터링:** Lightsail 2GB 환경에서 메모리 사용률을 모니터링하기 위해 서버 B에 `infra/monitor.sh` 스크립트가 마련되어 있습니다.

```
watch -n 5 ./infra/monitor.sh
```

명령으로 5초마다 ES Heap, Qdrant, Redis, 시스템 메모리 상태를 확인할 수 있습니다 ⁶⁸. 운영 중 메모리 부족 증상이 있다면 이 스크립트 출력을 참고하여 어떤 서비스가 많은 메모리를 차지하는지 판단합니다.

이러한 명령어들을 상황에 맞게 활용하면, 운영자가 직접 서비스 상태를 점검하고 필요한 조치를 명령줄에서 빠르게 취할 수 있습니다. 특히 **curl**과 **docker compose** 관련 명령은 자주 쓰이므로 숙달해 두는 것을 권장합니다.

9. 운영 로그 샘플 및 이상 탐지 기준

운영 로그는 시스템의 건강 상태와 이상 징후를 파악하는 중요한 수단입니다. 아래는 정상적인 요청 처리 시 나타나는 로그 예시와, 이를 토대로 한 이상 탐지 기준을 설명합니다:

정상 요청 처리 로그 예시

서버 A (FastAPI 애플리케이션)의 로그는 uvicorn 및 애플리케이션 로거를 통해 출력됩니다. 일반적으로 각 요청별로 접근 로그와 주요 이벤트 정보가 기록됩니다. 예를 들어, 어떤 사용자가 "치솔"을 질의했다고 가정하면 로그 흐름은 다음과 비슷합니다 (SAFE_MODE off, hybrid 모드 가정):

```
INFO:      172.26.0.1:56114 - "POST /v1/search HTTP/1.1" 200 OK
INFO [backend.logic.integrated_search] □ Hybrid search service initialized
(Elasticsearch + Qdrant)
INFO [backend.logic.integrated_search] Cache MISS: search results for ['치솔']
INFO [backend.logic.integrated_search] Selected intent=PRODUCT_SEARCH,
item="치솔"
INFO [backend.logic.integrated_search] Top3 results: 3 items (mode=hybrid,
```

```
safe_mode=False)
INFO [backend.logic.integrated_search] Timing: nlu=45ms, expand=20ms (cache hit), search=85ms, rerank=120ms, total=280ms
```

- 첫 줄은 **uvicorn**의 **access log**로, 클라이언트 요청에 대한 응답 정보를 보여줍니다 (IP, 메소드+경로, 상태코드, 응답시간 등).
- 이후 **애플리케이션 로그**에는 통합 검색 파이프라인 내 중요 이벤트들이 기록됩니다. 위 예에서 **"Hybrid search service initialized"** 는 애플리케이션이 처음 Elasticsearch+Qdrant 연결을 성공했음을 나타냅니다 ⁵². **"Cache MISS: search results for ['치솔']"** 은 이번 검색어에 대해 캐시에 결과가 없어 MISS되어 실제 검색을 수행했음을 뜻합니다 ⁴⁶.
- **"Selected intent=PRODUCT_SEARCH"** 부분은 NLU 단계에서 의도 분류 결과를 나타낸 것입니다. **"item=\"치솔\""** 은 추출된 주요 상품 키워드입니다. **SAFE_MODE**가 꺼져 있었다면 더 자세히 NLU 토큰 사용량 등이 로그되었을 수 있습니다.
- **"Top3 results: 3 items (mode=hybrid, safe_mode=False)"** 는 검색이 완료되어 3개의 top 결과를 얻었고, 현재 검색모드와 **safe_mode** 플래그 상태를 표시한 것입니다. (이 로그는 응답을 만들면서 개발용으로 찍었을 가능성이 높습니다.)
- 마지막 **"Timing: ... total=280ms"** 는 각 단계별 소요 시간과 전체 요청 처리 시간을 보여줍니다. NLU에 45ms, 키워드 확장에 20ms, 검색에 85ms, LLM rerank에 120ms 등이 걸려 전체 0.28초가 소요된 상황입니다. 이런 로그는 성능 모니터링에 유용합니다.

또한 Redis 캐시 초기 연결 시 **"Redis cache connected: redis://..."** 라는 로그가 한번 나타나며 ³², STT WebSocket 연결/종료 시에도 별도의 로그나 print가 나올 수 있습니다 (예: **"WebSocket connection accepted"** 등).

이상이 없는 경우 로그 상에는 **오류나 경고 없이** 위와 같은 **INFO 레벨 메시지**들만 주로 보입니다. 토큰 사용량도 **SAFE_MODE=false** 상태에서는 NLU/확장/LLM 단계 합계 토큰수가 **SearchResponse metadata**에 담기지만 로그에는 기본적으로 나오지 않습니다 (debug 모드에서는 나올 수도 있음). 요약하면 **정상 상태**의 로그 패턴은:

- 200 응답의 access log 연속 (에러 코드 드뭄)
- **Cache HIT/MISS** 로그에서 대부분 첫 질의 후 반복 질의에는 HIT 증가
- **Hybrid search service initialized** 는 초기 한번 나오고 이후 중복 없음 (hybrid 계속 사용 중이라는 뜻)
- **timing** 출력 값이 대체로 수백 ms 내외
- **safe_mode** 플래그는 의도한 값대로 표시 (운영 기본은 False)
- **selected intent** 가 대부분 **PRODUCT_SEARCH** 또는 **LOCATION** 등 유효값 (UNSUPPORTED가 빈출하지 않음)

이러한 로그들이 보이면 시스템은 건강하게 돌아가고 있는 것입니다.

이상 징후 탐지 기준

운영 로그나 응답을 모니터링하면서 다음과 같은 **이상 패턴**이 발견되면 조치가 필요합니다:

- **토큰 사용량 급증**: 평소 **SAFE_MODE**가 **True**인 환경에서는 모든 요청에서 **token_usage.total_tokens**가 0에 가까워야 합니다 ^{20 49}. 그런데 어느 순간부터 응답 메타데이터에 **total_tokens**가 상당히 큰 값(수백~수천 이상)으로 나타나거나, OpenAI API 호출 비용이 급증한다면 **SAFE_MODE** 설정이 꺼졌거나 외부 LLM 호출 빈도가 늘어난 것입니다. 이는 의도치 않은 설정 변경이나 트래픽 패턴 변화(모호한 질의 증가로 LLM 호출 증가)를 의미할 수 있습니다. **대응**: **SAFE_MODE** 설정을 재확인하고, 필요시 일시적으로 켜서 비용을 억제합니다. 또한 토큰 사용량 급증이 특정 기능(예: 특정 길이 질의로 인한 Clarification LLM 호출) 때문은 아닌지 로그를 분석합니다.

- **응답시간 지연 증가:** 로그상의 `timing_ms.total` 또는 access log의 처리 시간이 지속적으로 높아지는 추세는 성능 저하의 신호입니다. 예컨대 평균 300ms 이하이던 `/v1/search` 처리가 어느 날 800ms 이상으로 자주 보이는 경우입니다. 이때는 **어느 단계가 지연되고 있는지** `timing_ms` 세부치를 봅니다. `search` 단계 시간이 커지면 Elasticsearch 또는 Qdrant 성능 문제일 수 있고 `rerank` 단계가 커지면 외부 LLM 지연 또는 네트워크 문제가 있을 수 있습니다. 또는 캐시 HIT율이 낮아져서 매번 검색을 수행하는 상황일 수도 있습니다. **대응:** 해당 컴포넌트 상태(예: Elasticsearch CPU/Mem) 점검 및 재기동, 또는 일시적으로 `SAFE_MODE`를 켜 외부 지연 제거 등 원인별 대응을 합니다. 응답 지연은 사용자 경험에 직접 영향이 크므로, 슬랙 등 알림을 설정해 95퍼센타일 응답시간이 임계치를 넘으면 경고하도록 하는 것도 좋습니다.
- **검색 결과 없음 비율 증가:** 사용자의 검색 질의에 대해 `top3` 결과가 빈 경우, 또는 `is_in_scope=false`로 응답하는 케이스가 평소보다 늘어날 경우입니다. 이는 상품 DB 색인 누락, 검색엔진 인덱스 손상, 혹은 NLU가 질의를 잘못 `UNSUPPORTED`로 분류하는 등의 문제일 수 있습니다. 특히 모든 질의에 결과가 없다면 Elasticsearch/Qdrant 둘 다 동작 안 하거나 SQLite에도 검색이 안 되는 심각한 상태입니다. **대응:** 먼저 `/health`와 로그를 통해 Elasticsearch/Qdrant 적응 여부를 확인합니다. 문제가 없는데도 결과 없음이 많다면, 인덱스 문서 수나 상품 DB를 확인하여 누락이 없는지 봅니다. 필요 시 `indexer` 스크립트를 재실행하여 인덱스를 갱신하고, 그래도 해결되지 않으면 NLU 모듈 오작동(예: 모든 질의를 `OTHER`로 분류) 가능성을 고려해 `SAFE_MODE`를 켜서 로컬 NLU로 대체해보는 등 조치를 취합니다.
- **오류 로그 발생:** ERROR 또는 WARNING 레벨의 로그가 보이면 해당 내용이 무엇인지 즉시 조사합니다. 예를 들어 `"△ Hybrid search init failed"` ⁶³, `"Cache set error"` ⁷¹, `"Exception: Search failed: ..."` 등의 로그입니다. 이러한 로그는 앞서 언급한 구성요소 장애와 연결된 경우가 많으므로, 그에 맞춰 섹션 7의 가이드대로 대처하면 됩니다. 중요한 것은 **같은 경고/오류가 반복적으로 다수 발생**하는지를 보는 것입니다. 한번 발생하고 마는 것은 일시적일 수 있으나, 지속 반복되면 해당 기능이 망가졌음을 뜻합니다.
- **리소스 임계치 도달:** 로그 자체는 아니지만, `docker stats`나 `monitor.sh`로 관찰했을 때 컨테이너 메모리 사용량이 제한치에 근접한 경우입니다. 예를 들어 Elasticsearch 컨테이너가 1GB 한도에서 950MB 이상을 계속 쓰는 상황, Python API 컨테이너가 1.5GB 중 1.4GB 상시 사용 등입니다. 이 경우 OOM Kill 등의 위험이 있으므로, 사전에 인덱스 샤드 조정, Whisper 모델 다운스케일(이미 base로 낮췄지만) 등 최적화가 필요합니다 ^{60 72}. 또한 주기적 재시작(예: 심야에 한 번)으로 누수를 방지하는 것도 방법입니다.

종합하면, 운영 로그를 통해 평소와 다른 패턴이 감지되면 빠르게 원인을 분석하고 설정 변경이나 리소스 증설, 모드 전환 등의 대응을 해야 합니다. 특히 **토큰 사용량, 응답 지연, 결과 유효성** 세 가지는 사용 경험과 비용에 직결되므로 이상치를 놓치지 않아야 합니다.

운영자는 평소 **정상 로그의 모습**을 숙지하고 있어야 하며, 이 가이드에 제시된 기준들을 참고하여 이상 상황 발생 시 신속히 대처함으로써 서비스 중단이나 품질 저하를 최소화해야 합니다. ^{73 59}

이 문서는 **신규 인수자, 운영자, 유지보수 인력**이 다이스 AI 상품 위치 안내 서버를 재배포하거나 장애 대응, 상태 점검, 운영 정책 변경 등을 수행할 때 알아야 할 사항을 정리한 것입니다. 실무 현장에서 바로 활용할 수 있도록 구체적인 설정, 명령어, 점검 방법을 담고 있으며, 향후 서비스 변경 시 본 문서를 업데이트하여 최신 운영 절차를 유지하시기 바랍니다.

1 14 18 23 27 28 29 31 33 34 36 37 38 .env.example
<https://github.com/LeeYoungGeun/daiso-category-search/blob/98c804c69bee46ed50c832be57d148dc0bd4b687/.env.example>

2 6 7 10 11 12 15 40 67 68 73 **README.md**

<https://github.com/LeeYoungGeun/daiso-category-search/blob/98c804c69bee46ed50c832be57d148dc0bd4b687/README.md>

3 4 26 39 **docker-compose.app.yml**

<https://github.com/LeeYoungGeun/daiso-category-search/blob/98c804c69bee46ed50c832be57d148dc0bd4b687/infra/docker-compose.app.yml>

5 30 **docker-compose.data.yml**

<https://github.com/LeeYoungGeun/daiso-category-search/blob/98c804c69bee46ed50c832be57d148dc0bd4b687/infra/docker-compose.data.yml>

8 9 65 66 **start_data.sh**

https://github.com/LeeYoungGeun/daiso-category-search/blob/98c804c69bee46ed50c832be57d148dc0bd4b687/infra/start_data.sh

13 16 17 35 51 54 69 70 **hybrid.py**

<https://github.com/LeeYoungGeun/daiso-category-search/blob/98c804c69bee46ed50c832be57d148dc0bd4b687/backend/search/hybrid.py>

19 24 43 45 46 48 49 52 53 61 62 63 **integrated_search.py**

https://github.com/LeeYoungGeun/daiso-category-search/blob/98c804c69bee46ed50c832be57d148dc0bd4b687/backend/logic/integrated_search.py

20 21 22 25 **nlu.py**

<https://github.com/LeeYoungGeun/daiso-category-search/blob/98c804c69bee46ed50c832be57d148dc0bd4b687/poc/kms/nlu.py>

32 47 55 56 57 71 **cache.py**

<https://github.com/LeeYoungGeun/daiso-category-search/blob/98c804c69bee46ed50c832be57d148dc0bd4b687/backend/search/cache.py>

41 42 44 **main.py**

<https://github.com/LeeYoungGeun/daiso-category-search/blob/98c804c69bee46ed50c832be57d148dc0bd4b687/backend/main.py>

50 **loadtest_search.py**

https://github.com/LeeYoungGeun/daiso-category-search/blob/98c804c69bee46ed50c832be57d148dc0bd4b687/scripts/loadtest_search.py

58 **config.yaml**

<https://github.com/LeeYoungGeun/daiso-category-search/blob/cf5319975a26cfa87e3b605b2dd733407038ba1e/backend/config.yaml>

59 60 64 72 **CHANGELOG.md**

<https://github.com/LeeYoungGeun/daiso-category-search/blob/98c804c69bee46ed50c832be57d148dc0bd4b687/CHANGELOG.md>